BÁO CÁO NGHIÊN CỨU KHOA HỌC HÀNG TUẦN

Họ và tên sinh viên:	Nguyễn Việt Linh (23020832), Vũ Hàn Tín (23020874)
Tên đề tài nghiên cứu:	**Thiết kế phần cứng cho Depthwise Convolution**
Tuần báo cáo:	Tuần 6–7: 11/10/2026 – 23/10/2026
Người hướng dẫn:	TS Nguyễn Kiêm Hùng

1. Phân công công việc đã thực hiện

✔**Đọc tài liệu tham khảo.** MobileNets (Howard và cs., 2017) để nắm kiến trúc MobileNetV1 và kích thước đặc trưng qua các lớp; Bai, Zhao, Huang (2018) về bộ tăng tốc CNN trên FPGA dùng depthwise separable convolution (tổ chức bộ nhớ, các khối tính toán). **(Nguyễn Việt Linh & Vũ Hàn Tín)**

✔**Hệ thống hóa cơ sở lý thuyết.** Tích chập thông thường, depthwise convolution, pointwise convolution và depthwise separable convolution; công thức kích thước đầu ra, số weights, số phép MAC và tỉ lệ MAC giữa DSC với tích chập thông thường. **(Vũ Hàn Tín)**

✔**Xác định baseline thiết kế và quy trình kiểm chứng.** Python (golden model) → RTL SystemVerilog → testbench → so sánh với golden model → tổng hợp trên FPGA; liệt kê các ca kiểm thử biên. **(Vũ Hàn Tín)**

✔**Chọn lớp DWC tham chiếu.** Lớp DWC đầu tiên của MobileNetV1 (112×112×32, kernel 3×3, stride 1); tính khối lượng tính toán và dung lượng dữ liệu của lớp này. **(Nguyễn Việt Linh)**

✔**Đề xuất sơ bộ kiến trúc phần cứng.** Các khối chức năng, luồng dữ liệu, các phương án tính toán và định dạng số (signed INT8, bộ tích lũy 32-bit). **(Nguyễn Việt Linh)**

2. Kết quả đạt được

Trong tuần 6–7, nhiệm vụ chính là xây dựng cơ sở lý thuyết toán học về các phép tích chập và xác định baseline thiết kế chung. Các kết quả chính được trình bày dưới đây.

2.1. Tích chập thông thường (CNN)

Tích chập dùng một kernel nhỏ trượt trên ma trận đầu vào; tại mỗi vị trí, các phần tử của kernel được nhân với các phần tử ảnh tương ứng rồi cộng lại thành một giá trị của feature map. Với ảnh nhiều kênh, đầu vào là tensor X có kích thước $H_{in}\times W_{in}\times C_{in}$ (ảnh RGB có C_{in}=3) và mỗi kernel có kích thước $K\times K\times C_{in}$. Mỗi kernel quét qua toàn bộ các kênh đầu vào cùng lúc để tạo ra duy nhất một kênh đầu ra; muốn có C_{out} kênh đầu ra cần C_{out} kernel:

$$Y(h,w,m)=\sum_{c=0}^{C_{in}-1}\sum_{i=0}^{K-1}\sum_{j=0}^{K-1}X(h+i,\,w+j,\,c)\cdot W(i,j,c,m)$$

Trong báo cáo này chỉ xét stride = 1 và không padding, tensor đầu ra Y có kích thước $H_{out}\times W_{out}\times C_{out}$:

$$H_{out}=H_{in}-K+1,\qquad W_{out}=W_{in}-K+1$$

Chi phí của tích chập thông thường:

- Số weights: $K^2\cdot C_{in}\cdot C_{out}$
- Số phép MAC (bộ nhân–cộng): $H_{out}\cdot W_{out}\cdot K^2\cdot C_{in}\cdot C_{out}$

Cách phân tích: $MAC=C_{out}\cdot H_{out}\cdot W_{out}\cdot(K^2\cdot C_{in})$, trong đó $K^2\cdot C_{in}$ là số MAC cho một điểm ảnh của Y, nhân với $H_{out}\cdot W_{out}$ được số MAC cho một lát cắt (một kênh) của Y, và nhân với C_{out} được tổng số MAC của toàn bộ tensor.

2.2. Depthwise convolution và depthwise separable convolution

Depthwise convolution (tích chập theo chiều sâu) thực hiện tích chập độc lập trên từng kênh: mỗi kênh đầu vào dùng một kernel $K\times K$ riêng, nên số kênh đầu ra bằng số kênh đầu vào ($C_{out}=C_{in}$):

$$Y_{DW}(h,w,c)=\sum_{i=0}^{K-1}\sum_{j=0}^{K-1}X(h+i,\,w+j,\,c)\cdot W_{DW}(i,j,c)$$

Tensor đầu ra Y_{DW} có kích thước $H_{out}\times W_{out}\times C_{in}$, với H_{out}, W_{out} tính như trên. Số weights là $K^2\cdot C_{in}$ và số phép MAC là $H_{out}\cdot W_{out}\cdot C_{in}\cdot K^2$.

Depthwise convolution chỉ lọc theo không gian trên từng kênh mà không trộn thông tin giữa các kênh. Bước trộn kênh thường được thực hiện bằng pointwise convolution (tích chập 1×1), đồng thời cho phép chọn số kênh đầu ra C_{out}. Pointwise convolution không thuộc phạm vi thiết kế của đề tài, chỉ được dùng để so sánh chi phí; với C_{out} kernel kích thước $1\times1\times C_{in}$:

$$Y_{PW}(h,w,m)=\sum_{c=0}^{C_{in}-1}Y_{DW}(h,\,w,\,c)\cdot W_{PW}(c,\,m)$$

Ghép depthwise và pointwise convolution ta được depthwise separable convolution (DSC). Bảng 1 so sánh chi phí của các phép tích chập.

Bảng 1. So sánh số weights và số phép MAC (stride = 1, không padding)

Phép tích chập	Số weights	Số phép MAC
Tích chập thông thường	$K^2\cdot C_{in}\cdot C_{out}$	$H_{out}\cdot W_{out}\cdot K^2\cdot C_{in}\cdot C_{out}$
Depthwise (DW)	$K^2\cdot C_{in}$	$H_{out}\cdot W_{out}\cdot C_{in}\cdot K^2$
Pointwise (PW)	$C_{in}\cdot C_{out}$	$H_{out}\cdot W_{out}\cdot C_{in}\cdot C_{out}$

Phép tích chập	Số weights	Số phép MAC
DSC = DW + PW	$K^2 \cdot C_{in} + C_{in} \cdot C_{out}$	$H_{out} \cdot W_{out} \cdot C_{in} \cdot (K^2 + C_{out})$

Từ Bảng 1, tỉ lệ số MAC của DSC so với tích chập thông thường (cùng kích thước đầu vào và đầu ra) là:

$$\frac{\text{MAC}_{\text{DSC}}}{\text{MAC}_{\text{CNN}}} = \frac{1}{C_{out}} + \frac{1}{K^2}$$

Ví dụ kiểm tra với tensor 112×112×32, K=3, C_{out}=32: tích chập thông thường cần 115 605 504 MAC; DSC cần 16 457 728 MAC (3 612 672 cho DW và 12 845 056 cho PW). Tỉ lệ là 0,1424 = 1/32 + 1/9, tức giảm khoảng 7 lần.

2.3. Lớp DWC tham chiếu và khối lượng tính toán

Theo kiến trúc MobileNetV1, ảnh đầu vào 224×224×3 đi qua lớp convolution đầu tiên (stride 2) cho tensor đặc trưng 112×112×32; lớp depthwise convolution đầu tiên xử lý tensor này bằng kernel 3×3, stride 1. Lớp này được chọn làm bài toán tham chiếu (Bảng 2).

Bảng 2. Thông số lớp DWC tham chiếu

Tham số	Giá trị
Kiến trúc tham chiếu	MobileNetV1
Vị trí	Lớp DWC đầu tiên, sau lớp convolution ban đầu
Kích thước đầu vào	112 × 112 × 32
Kernel / Stride / Channel multiplier	3 × 3 / 1 / 1
Padding đề xuất	SAME (zero padding)
Kích thước đầu ra	112 × 112 × 32
Định dạng dữ liệu (đề xuất)	Signed INT8
Bộ tích lũy (đề xuất)	Signed 32-bit

Khối lượng tính toán và dung lượng dữ liệu của lớp tham chiếu (giả định dữ liệu và trọng số là signed INT8):

Bảng 3. Khối lượng tính toán và dung lượng dữ liệu

Đại lượng	Cách tính	Giá trị
Số phần tử đầu ra	112 × 112 × 32	401 408
Số MAC cho mỗi phần tử đầu ra	3 × 3	9
Tổng số phép MAC	401 408 × 9	3 612 672 (≈ 3,61 triệu)
Tensor đầu vào (INT8)	112 × 112 × 32 byte	401 408 byte (≈ 392 KiB)

Đại lượng	Cách tính	Giá trị
Tensor trọng số (INT8)	3 × 3 × 32 byte	288 byte
Tensor đầu ra (nếu lưu INT8)	112 × 112 × 32 byte	401 408 byte

- Các con số chỉ tính dữ liệu thô, chưa gồm bias, metadata, bộ đệm trung gian; input và output cũng không nhất thiết phải lưu đồng thời toàn bộ trong phần cứng, tùy kiến trúc.
- 3 612 672 là số phép toán toán học, không phải số chu kỳ thực thi. Nếu chỉ có một đơn vị MAC, mỗi phần tử đầu ra cần tối thiểu 9 lần MAC (chưa tính thời gian nạp dữ liệu, điều khiển và ghi kết quả).

2.4. Baseline thiết kế và kế hoạch kiểm chứng

Để đơn giản hóa quá trình thiết kế mà không mất tính tổng quát, baseline quy ước: input là tensor $H \times W \times C$, kernel 3×3, stride = 1, không padding, bias = 0. Quy trình thực hiện:

1. **Mô phỏng Python:** sinh input, kernel và tính kết quả chuẩn (golden model).
2. **Thiết kế RTL:** mạch tính depthwise convolution bằng SystemVerilog.
3. **Testbench/simulator:** đưa input và kernel vào RTL, thu output; bao phủ các ca kiểm thử ở Bảng 4.
4. **So sánh** output của RTL với golden model.
5. **Tổng hợp, implementation và kiểm thử trên FPGA:** ghi nhận latency và throughput (số pixel xử lý mỗi giây).

Bảng 4. Các nhóm ca kiểm thử dự kiến

Nhóm	Nội dung kiểm thử
Phép tính cơ bản	Input và kernel toàn 0; chỉ 1 pixel (hoặc 1 điểm kernel) khác 0; input = 1, weight = 1; giá trị âm/dương xen kẽ; giá trị min/max của INT8; dữ liệu khác nhau giữa các kênh (phát hiện nhầm chỉ số kênh).
Cửa sổ và padding	Vị trí đầu tiên, các góc, các cạnh, cuối hàng, cuối tensor; phần tử ngoài biên xử lý theo zero padding.
Tham số	Cấu hình tham chiếu (3×3, stride 1, SAME); tensor nhỏ với padding VALID; nhiều kênh để xác nhận tính độc lập giữa các kênh.
Giao tiếp và điều khiển	Reset; tín hiệu bắt đầu; thời điểm dữ liệu đầu ra hợp lệ (valid); tín hiệu hoàn thành; bắt đầu phép tính mới sau khi phép tính trước kết thúc.
Ngẫu nhiên	Sinh ngẫu nhiên input và kernel, đối chiếu toàn bộ đầu ra với golden model.

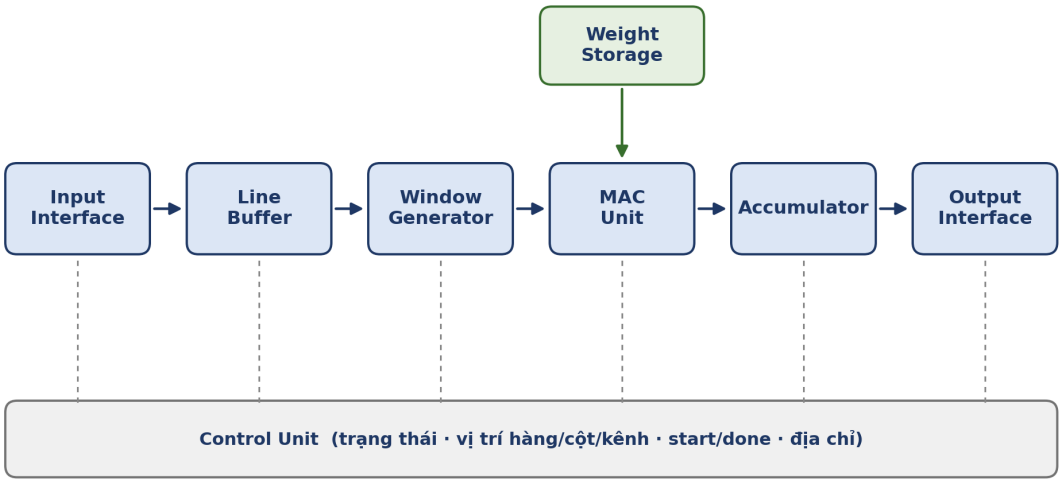
Tiêu chí đúng: output RTL khớp chính xác từng phần tử với golden model (cùng quy tắc số học); đồng thời phải kiểm tra số lượng phần tử đầu ra, thứ tự xuất dữ liệu và tín hiệu hoàn

thành. Stride 2 và padding VALID là các ca mở rộng, không thay đổi cấu hình lớp tham chiếu chính.

Chỉ tiêu đánh giá dự kiến: độ chính xác chức năng; latency; throughput; số chu kỳ; tần số hoạt động; tài nguyên FPGA (LUT, flip-flop, DSP, block RAM) nếu có điều kiện tổng hợp.

2.5. Đề xuất sơ bộ kiến trúc phần cứng

Kiến trúc đề xuất gồm các khối chức năng ở Bảng 5; luồng dữ liệu tổng quát: Input Interface → Line Buffer → Window Generator → MAC Unit → Accumulator → Output Interface (Hình 2). Weight Storage cấp trọng số cho MAC Unit; Control Unit điều khiển các bước xử lý và cập nhật địa chỉ.



Hình 2. Sơ đồ khối sơ bộ của module DWC (đề xuất; sẽ được chi tiết hóa ở tuần 8–9).

Bảng 5. Chức năng các khối

Khối	Chức năng
Input Interface	Tiếp nhận dữ liệu đầu vào theo thứ tự xác định.
Line Buffer	Lưu các dòng dữ liệu cần thiết để tạo cửa sổ 3×3.
Window Generator	Tạo cửa sổ 3×3 tương ứng với vị trí đầu ra hiện tại.
Weight Storage	Lưu 9 trọng số cho mỗi kênh, tổng cộng 288 trọng số INT8.
MAC Unit	Thực hiện phép nhân và cộng tích lũy.
Accumulator	Lưu kết quả trung gian với độ rộng đủ lớn.
Output Interface	Xuất kết quả đầu ra theo thứ tự quy định.
Control Unit	Quản lý trạng thái, vị trí hàng/cột/kênh, tín hiệu bắt đầu và kết thúc.

Các phương án tính toán sẽ được cân nhắc:

- **Một MAC dùng chung:** ít tài nguyên tính toán nhưng cần nhiều chu kỳ hơn; đơn giản cho RTL và kiểm chứng nên dùng làm bước đầu.
- **Nhiều MAC song song:** tăng thông lượng, đổi lại tốn nhiều DSP/logic hơn.

- **Streaming kết hợp line buffer:** giảm nhu cầu lưu toàn bộ feature map và tận dụng dữ liệu dùng lại giữa các cửa sổ liên tiếp.

Chưa thể kết luận phương án nào tốt hơn; cần so sánh số chu kỳ, tần số và tài nguyên tổng hợp trên cùng một cấu hình dữ liệu.

Định dạng số: đề xuất signed INT8 cho đầu vào và trọng số. Tích của hai số signed 8-bit biểu diễn được bằng 16 bit, nhưng tổng 9 tích có thể vượt phạm vi này (trường hợp xấu nhất $9 \times 128 \times 128 = 147\,456$, cần tối thiểu 19 bit có dấu), nên bộ tích lũy cần rộng hơn; thiết kế cơ sở đề xuất signed 32-bit để có khoảng an toàn.

Tài liệu tham khảo

[1] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017.

[2] L. Bai, Y. Zhao, X. Huang, "A CNN Accelerator on FPGA Using Depthwise Separable Convolution," 2018.

3. Khó khăn, vướng mắc

Chưa có

4. Kế hoạch tuần tiếp theo

Công việc dự kiến (tuần 8–9):

- **Xây dựng sơ đồ kiến trúc phần cứng tổng thể:** chi tiết hóa các khối, luồng dữ liệu, tín hiệu điều khiển và thứ tự lưu tensor.
- **Viết golden model bằng Python theo baseline:** sinh input/kernel, tính kết quả chuẩn và sinh các ca kiểm thử biên đã liệt kê.

Mục tiêu mong muốn:

- Golden model chạy đúng, dùng làm chuẩn đối chiếu cho RTL.
- Sơ đồ kiến trúc đủ rõ ràng để bắt đầu viết RTL SystemVerilog ở giai đoạn tiếp theo.

5. Checklist theo dõi tiến độ công việc

STT	Nội dung công việc	Deadline dự kiến	Trạng thái (✔/✘)	Ghi chú
1	Tìm và đọc tài liệu tham khảo	23/102026	✔	MobileNets (2017); Bai và cs. (2018)
2	Viết tổng quan nghiên cứu (Literature Review)	23/102026	✔	Báo cáo nghiên cứu tuần 6-7
3	Viết golden model bằng Python	Tuần 8–9	✘	Kế hoạch tuần tiếp theo
4	Hoàn thiện sơ đồ kiến trúc phần cứng tổng thể	Tuần 8–9	✘	Kế hoạch tuần tiếp theo

Ngày: 23/10/2026 **Sinh viên ký tên:** Nguyễn Việt Linh, Vũ Hàn Tín